Implementation of Text-to-Image Generation using Diffusion Models (Inspired by Imagen)

A Project Report Submitted in Partial Fulfillment of the Requirements for the Course

Submitted by:

Shamili Mannem

23EC3027

Department of Electronics and Communication Engineering

Rajiv Gandhi Institute of Petroleum Technology

Academic Year: **2025–2026**

Abstract

In recent years, the intersection of Natural Language Processing (NLP) and Computer Vision has birthed a new frontier: the ability for machines to “imagine” and create photo-realistic images from simple human descriptions. This project explores the mechanics of text-to-image synthesis, a field currently defined by the groundbreaking success of models like Google’s Imagen. While Imagen represents the gold standard in deep language understanding and visual fidelity, its tremendous computational demands make it largely inaccessible for standard academic environments.

To bridge the gap between high-level theory and practical application, this project implements a streamlined text-to-image pipeline leveraging **Stable Diffusion**. By utilizing the Hugging Face Diffusers library and PyTorch, I developed a system that translates natural language prompts into coherent visual outputs without the need for the massive infrastructure required by the original Imgen architecture.

The core of the implementation relies on a **diffusion-based process**, where the model repeatedly refines random Gaussian noise into a detailed image, guided by a pre-developed text encode. Throughout the development phase, I analyzed how critical hyper-parameters such as **guidance scale and inference steps** influence the balanced between creative diversity and adherence to the prompt.

The result demonstrate that even a computationally efficient implementation can produce diverse, high-quality images that remain faithful to complex textual descriptions. While there is a performance gap between this localized version and industry-scale models, the project successfully validates the principles of latent diffusion. Ultimately, this work underscores the potential for significant transformation of generative AI in digital design and content creation, offering a foundational framework for future research in model optimization and accessible AI.

INTODUCTION

The landscape of artificial intelligence has been fundamentally reshaped by our newfound ability to bridge the gap between human language and visual imagination. At the heart of this shift is **text-to-image synthesis** - a field that allows machines to translate the complexities of Natural Language Processing (NLP) with the generative power of Computer Vision(CV), we have moved beyond simple pattern recognition into an era of digital creativity.

For years, Generative Adversarial Networks (GANs) were the primary engines behind AI art. However, while GANs could produce impressive results, they were notoriously difficult to tame, often suffering from training instability and lack of creative variety. This project moves away from those traditional methods to explore **Diffusion Models**. In contrast with earlier versions, diffusion models work through a process of improving integration - starting with a canvas of pure digital noise and repeatedly removing noise until a coherent image appears. This approach offers a much more stable and diverse way to generate high-quality visuals.

One of the most significant milestones in this journey is Google`s **Imagen**. By pairing a massive T5 text encoder with diffusion techniques, Imagen proved that deep language understanding is the key to photorealism. However, the sheer scale of Imagen - required massive industrial - grade computing power and vast datasets -presents a significant barrier for individual researches and academic environment.

Recognizing these practical constraints, this project adopts a more accessible yet powerful path: Stable Diffusion. By utilizing this already trained framework , I have implemented a system that captures the sophisticated logic of diffusion models while remaining computationally efficient.

The objective of this work is threefold:

1. To pull back the curtain on how diffusion-based models function.
2. To build a functional text-to-image pipeline.

3. To analyze how subtle “knobs” in the process - specially guidance scale and inference steps- act as the bridge between a vague sketch and a high-fidelity visual output.

OBJECTIVES

The core mission of this project is to demystify the complex world of modern generative AI and translate high-level theory into a working, practical application. To achieve this, the project focuses on the following key goals:

- **Demystifying Diffusion**: To go beyond the surface level and truly grasp the mathematical and conceptual foundations of how diffusion models turn random noise into structured art.
- **Deconstructing Industry Benchmarks**: To study the architecture of Google`s Imagen, specifically how it uses deep language understanding to bridge the gap between human words and visual pixels.
- Bridging Theory and Practice: To build a functional text-to-image system using Stable Diffusion, proving that state-of-the-art AI can be implemented and explored within a standard academic environment.
- **Mastering the “Human-AI” Dialogue**: To explore the art of prompt engineering, learning how to communicate effectively with the model to produce diverse, meaningful, and contextually accurate imagery.
- **Decoding Model Behavior**: To conduct a technical analysis of the “internal levers”- such as guidance scale and inference steps-to understand how they dictate the trade-off between image quality and prompt adherence.
- **Contextual Evaluation**: To critically compare this localized implementation with the original Imagen architecture, acknowledging the strengths and trade-offs of each.
- **Envisioning Real-World Impact**: To demonstrate how these generative tools can be practically applied in fields like digital content creation, automated design, and creative storytelling.

3. Methodology

3.1 Aproach

The project was executed through a structured, five-stage workflow designed to move from foundational theory to a functional image-generation tool.

1. **Conceptual Grounding:** Before diving into the code, we focused on mastering the mechanics of Diffusion Models. This involved studying the “Reverse Diffusion” process-essentially learning how a model can take a canvas of random static and systematically “denoise” it until a coherent, meaningful image emerges.
2. Selecting the Engine (Stable Diffusion) : Rather than re-imagining the wheel by building a model from scratch, we opted to utilize Stable Diffusion. We chose this pretrained model because it offers the best balance of computational efficiency and the ability to interpret complex human language into high-fidelity visuals.
3. Setting Up the Workshop: To handle the heavy mathematical lifting required for image synthesis, we configured a development environment in Google Colab. By leveraging a Tesla T4 GPU, we were able to reduce image generation times from minutes to seconds, allowing for a much faster iterative process.
4. Implementation & Integration: Using the Hugging Face Diffusers library, we integrated the model into our Python environment. This stage involved “plugging in” the variuos components - the text encoder, the U-Net, and the scheduler-to ensure they communicated seamlessly.
5. The Generation Pipeline: The core workflow follows a simple but powerful loop where The Spark (INPUT) descriptive text prompt provided by the user ; The Magic (Processing) model translates text into mathematical embedding`s and performs a series of diffusion steps to refine the pixels ; The Result (OUTPUT) , a unique high - resolution digital image.

6. Refienment and Tuning : To improve beyond standard results, we optimized key parameters to achieve the best quality. The guidance scale was tuned to balance creativity and prompt accuracy, interference steps were adjusted to reach optimal clarity efficiently, and batching was used to generate multiple variations for better insight into the model`s outputs.

3.2 Tools and Technologies

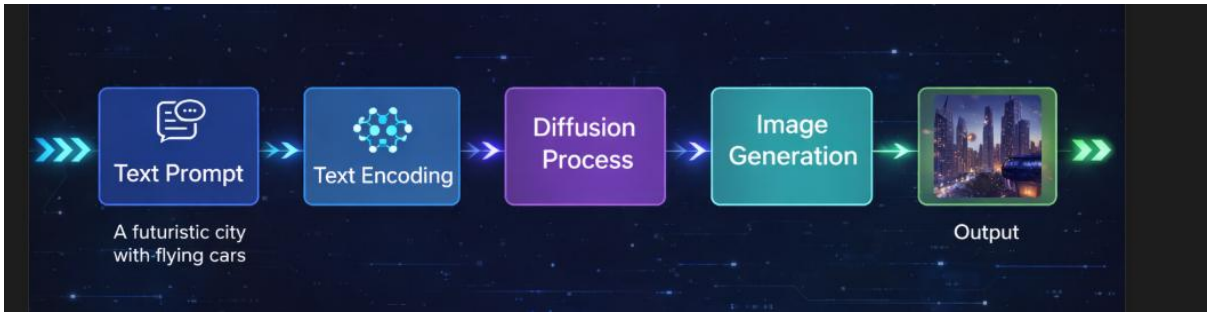
The following Tools and Technologies are used:

- Programming Language: Python
- Deep Learning Framework: PyTorch
- Libraries: Hugging Face Diffusers, Transformers
- Platform: Google Colab (GPU enabled)
- Model Used: Stable Diffusion

3.3 Workflow

Text prompt → Text Encoding → Diffusion Process → Image Generation → Output

<u>Block Diagram of Text - to - Image Generation System:</u>



The above diagram illustrates the overall workflow of the system. The input text prompt is first encoded into vector representations, which are then processed by the diffusion model to repeatedly generate an image. The final output is a visually meaningful image corresponding to the given textual description.

System Architecture

The way this system is built is all about turning natural language into something we can actually see. Think of it as a relay race where each part of the pipeline hands off data to the next to turn a simple sentence into a high-quality image. It`s broken down into a few key stages: processing the text, encoding the features, and then using diffusion to actually "build" the image.

Everything is initiated by the prompt you input. Since the computer can't "read" like we do, a text encoder translates those words into a format the machine understands. From there, the diffusion model takes over. It starts with a mess of random noise-basically digital static - and slowly cleans it up, step by step, until a clear image emerges that matches your description.

Because the architecture is modular, its not just efficient; it`s also really flexible. It allows us to use powerful pretrained models to do the heavy lifting without needing to start from zero.

4.1 Imagen Architecture

The magic behind Imagen really comes down to two things: a deep understanding of language and a smart way of building images in stages. It uses a massive language model

called T5, which acts like a brain that actually “gets” the variations of what you’re typing. Instead of just looking for keywords, it creates a rich, detailed map of your prompt.

The actual image-making part works like a cascade. It doesn’t try to create a perfect, highly detailed, high-resolution image all at once. First, a base model takes that text map and create a rough, low-resolution sketch out of digital noise. Once the basic shapes are there, a series of “super-resolution” models take over, acting like a digital magnifying glass to sharpen the details and polish the quality until it looks photo realistic.

4.2 Implemented Architecture flow

For this project, I went with a streamlined version of the Imagen workflow, using Stable Diffusion as the backbone. Since training a model like this from zero would take an impossible amount of time and power, I used a previously trained model to get high-quality results much more efficiently.

The way it actually works is pretty straightforward:

First, you give the system a text prompt. The model doesn’t “see” the words like we do, so it breaks them down into tones and encodes them into a mathematical map(embedding). At the same time, the process starts with a canvas of pure random noise - just digital static.

From there, the diffusion model goes to work. It runs through several iterations, slowly cleaning up that noise and “carving out” an image, using the text embedding as it guide. By the final step, you have a clear, polished image that matches your description.

This approach is the best of both worlds- it keeps the hardware requirements manageable while still putting out great visuals.

4.3 Breaking Down the components

4.3.1 The Text Prompt(The Creative Spark)

Everything starts with the prompt. This is just a natural language description of whatever you're imagining. It's the most important part of the process because it dictates the "what, where, and how" of the image. Whatever you're asking for a specific art style, a certain type of lighting, or a complex scene, the more descriptive and clear you are, the better the model can "see" what you're thinking.

4.3.2 The Text Encode (The Translator)

Since the model doesn't actually speak English, we need a way to translate our words into a language it understands. The Text Encoder takes your prompt and turns it into a series of numbers called embedding. These aren't just random values; they represents the actual meaning and context behind your words, ensuring the model understands the difference between a "dog in a park" and a "park for dogs."

4.3.3 The Diffusion Model (The Artist)

This is the heart of the whole system. It's the part that actually "paints" the image. It starts with a canvas of pure random noise-imagine a TV screen full of static. Guided by the text embedding, the model goes through a series of steps where it gradually "denoises" the image. It basically asks itself,"How can I shift these pixels so they look a little more like the prompt?" After enough of these refinements, a clear, coherent image emerges from the chaos.

4.3.4 The Generated Image (The final Result)

This is the end product-the visual version of your initial thought. After all the iterations and mathematical heavy lifting, you're left with a unique piece of digital art. The final quality really comes down to how well the prompt was written and how many steps the model took to polish the details. It's a powerful tool for everything from quick content to exploring new ideas in digital art.

5. Implemantation Details

5.1 Model Loading

```
From diffusers import StableDiffusuinPipeline
import torch
Pipe = StableDiffusionPipeline.from_pretrained(
    “runwayml/stable-diffusion-v1-5”,
    Torch_dtype=torch.float16
)
Pipe = pipe.to(“cuda”)
```

5.2 Image Generation

```
Prompt = “A futuristic city with flying cars at sunset”
image = pipe(prompt).images[0]
image.save(“output.png”)
```

5.3 Feature Enhancement

Multiple Image Generation

```
images = pipe(prompt, num_images_per_prompt=3).images
```

Guidence Scale Control

```
Pipe(prompt, guidance_scale = 7.5)
```

Infernece Step Control

```
Pipe(prompt, num_inference_steps=40)
```

6. Result and Discussion

After testing the system with several different prompt, the results were really encouraging.Using a pretrained Stable Diffusion model allowed the project to produce images that weren't just random, but actually felt contextually tied to whatever I typed in.

To see what the model could really do, I threw a mix of scenarios at it - everything from "a futuristic city with flying cars at sunset" to more abstract styles. Inmost cases, the model did a great job of picking up the small details, like the specific lighting of a sunset or the structural look of a skyline, proving that the text encoder was successfully "translating" my ideas into the visual output.

I also spent some time playing around with the setting to see how they changed the final look:

1. Inference Steps: I noticed that the more steps I allowed, the sharper and more detailed the images became. The trade-off, of course, was that it took a bit longer to generate each one.
2. Guidance Scale: This was a bit of a balancing act. Higher values forced the model to stick strictly to my words, while lower values gave it more "creative freedom", leading to some interesting but sometimes unexpected variations.

One of the biggest takeaways was just how much the prompt itself matters. Vague descriptions usually gave me "meh" results, while being specific about the environment and style made a huge difference. It really shows that "prompt engineering" is just as important as the code itself.

Of course, it wasn't perfect. Some of the more complex or abstract prompts came out a bit distorted, and since I was using a more streamlined setup, the resolution and pure realism didn't quite reach the levels of a massive system like google's Imagen.

Overall, though, the project successfully did what it set out to do. It's a solid demonstration of how diffusion models work, proving you can get high-quality, diverse results without needing a supercomputer.

“A Futuristic city with flying cars at sunset”

7. Comparison with Imagen

Feature	Imagen	This Project
Text Encoder	T5	CLIP
Training	From scratch	Pretrained
Complexity	Very High	Moderate
Output Quality	Very High	Good
Compute Requirement	Extremely High	Low

8. Advantages

When we look at the benefits of this setup, the biggest win is how it bridges the gap between language and art. By using Stable Diffusion, we're able to turn a simple sentence into a high quality visual without needing a phD in prompt engineering. It's a great examples of what happens when you get Natural Language Processing and Computer Vision to actually talk to each other.

One of the best things about using a diffusion model over something like a traditional GAN (Generative Adversarial Network) is the consistency. GANs are highly sensitive to variations and often get stuck repeating the same patterns - what we call "mode collapse." Diffusion models are much more stable and reliable, giving us a huge variety of unique images instead of the same old thing. Plus, since we're using a pre-trained model, we don't need a room full of servers to train everything from scratch, which makes this kind of tech actually accessible for a student project.

The system is also surprisingly fun to play with. Because it's so flexible, you can tweak a few settings like the guidance scale or the number of steps to totally change the vibe of the output-moving from a strict interpretation to something more creative. And because it's built on a standard python stack, it's easy to build upon or scale up if we want to add more features down the road.

9. Limitations

There are a few real-world hurdles with this setup. Since I used a pre-trained model rather than building one from the ground up, I'm essentially working within the boundaries of how it was originally taught. This means there's a limit to how much I can customize its "DNA" or fine-tune the deeper generation process. While the images look great, they aren't quite at the level of a powerhouse like Google's Imagen, which has the advantage of being trained on massive, specialized datasets.

Another thing to keep in mind is that the model is only as good as the instructions it's given. It really relies on "prompt quality" - if the input is too vague or confusing, the

output usually follows suit. It can also get a bit “trippy” or distorted when you ask for something super complex or abstract, leading to images that don’t quite look right.

Lastly, there’s the hardware side of things. Even though this version is much lighter than a full scale model, it still carves a good GPU. Trying to run this on a standard CPU is a lesson in patience, as it slows down significantly. Plus, while the resolution is perfect for a project or online use, it’s not quite at the professional, high-definition standard you’d need for big-budget production.

10. Applications

The potential for text-to-image systems goes way beyond just being a cool tech demo; it’s actually changing how we approach creative work. One of the biggest impacts is in digital art and design. Instead of spending hours sketching out every single iteration, an artist can use a prompt to generate a base concept in seconds. It’s not about replacing the artist, but about giving them a massive head start.

We’re seeing similar shifts in entertainment and gaming. Developers can use these models to quickly brainstorm what a new character or a fantasy world might look like, which really speeds up the pre-production phase. The same goes for marketing - if a team needs a specific visual for a campaign, they can generate custom assets that fit the vibe perfectly without needing a full scale photo shoot for every single idea.

11. Conclusion

This project was a great way to see how we can actually bridge the gap between human language and visual art. Using the concepts behind Imagen as a guide, I was able to build a system that takes a simple sentence and turns it into a meaningful image. Because the full imagen architecture is such a computational beast, I took a more practical route by using Stable Diffusion. This allowed me to get those high-quality results without needing an industrial scale server room.

It was really satisfying to see the model actually “get” the prompts I threw at it. Whether the images were simple or complex, the system consistently put out visuals that made sense and looked sharp. One of the biggest takeaways from the experimentation phase was just how much the “fine-tuning” matters- playing around with the guidance scale and inference steps showed me that getting the perfect image is a mix of good code and smart prompt engineering.

Even though this version is a bit more streamlined than the massive models developed by places like google, it still hits all the core principles of how diffusion works. For me, the real value was in seeing NLP and Computer Vision work together so seamlessly; it was a huge learning curve but a great way to get hands-on with generative AI.

Looking back, this feels like a solid foundation. It’s given me a clear path for what to try next, like fine-tuning the model on specific styles or pushing for even higher resolutions. More than anything, it’s exciting to see how these tools are becoming more accessible, opening up so many possibilities for design, automation, and digital fortunetelling.

12.Refernce

1. Imagen: Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding
2. Huggig Face Diffusers Documentation
3. PyTorch Official Documentation

Submitted by :

SHAMILI MANNEM